

Problem Description

A and B are playing a game. In this game, both of them are initially provided with a **list of n numbers**. (Both have the same list but their own copy).

Now, they both have a different strategy to play the game. A picks the element from **start of his list**. B picks from the **end of his list**.

You need to generate the result in form of an output list.

Method to be followed at each step to build the output list is:

1. If the number picked by A is **bigger than** B then this step's **output is 1** . B **removes** the number that was picked from their list.
2. If the number picked by A is **smaller than** B then this step's **output is 2** . A **removes** the number that was picked from their list.
3. If both have the **same number** then this step's **output is 0** . **Both A and B remove** the number that was picked from their list.

This game **ends** when at least one of them has no more elements to be picked i.e. when the **list gets empty**.

Output the built output list.

Constraints

$$1 \leq N \leq 10^6$$

$$1 \leq \text{num} \leq 10^9$$

Input format:

First line consists of a number n , size of the list provided.

Next line consists of n numbers separated by space.

Output format:

Output the required output list.

Logical Test Cases

Test Case 1

INPUT (STDIN)

3
1 2 3

EXPECTED OUTPUT

2 2 0

Test Case 2

INPUT (STDIN)

5
5 7 1 2 3

EXPECTED OUTPUT

1 1 1 2 0 2 2 2

Mandatory Test Cases

Test Case 1

KEYWORD

if(a[i]>a[j])

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

160

Test Case 3

NLOC

36

```
Stack1.c +
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1) return 0;
6     long long a[n], b[n];
7     for (int k = 0; k < n; k++) {
8         scanf("%lld", &a[k]);
9         b[k] = a[k];
10    }
11
12    int i = 0, j = n - 1;
13    while (i < n && j >= 0) {
14        if (a[i] > a[j]) {
15            printf("1 ");
16            j--;
17        } else if (a[i] < a[j]) {
18            printf("2 ");
19            i++;
20        } else {
21            printf("0 ");
22            i++;
23            j--;
24        }
25    }
```

main | CH.SC.U4CSE25225 | Stack1.c

Ln 29, Col 1 Spaces: 4 UTF-8 CRLF C

```
26  printf("\n");  
27  return 0;
```

28}

29

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3

1 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack1.exe

2 2 0

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

5 7 1 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack1.exe

1 1 1 2 0 2 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Stack	Question Information	Level 1	Challenge 42
----	---------	-------	----------------------	---------	--------------

Question description

Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.

the task is to implement stack operations for two stacks and merge the stacks into one.

Get two sets of stack elements and reverse them, then merge them into one stack.

Rajinikanth has given the deadline of only 15 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit?

Function Description

- a) push(): Adds the new item at the beginning of the linked list using the first pointer.
- b) pop(): Removes an item from the beginning using the first pointer.
- c) merge(): Links the first pointer second stack as next of the last pointer of the first list.

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed

The next line indicates the n number stack elements

Output Format:

First line indicates the top of the element of the stack

the second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

5 8
9 8 7 6 5
5 6 7 8 3 11 2 3

EXPECTED OUTPUT

5 6 7 8 9 3 2 11 3 8 7 6 5

Test Case 2

INPUT (STDIN)

10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14

EXPECTED OUTPUT

13 18 17 16 15 15 16 17 18 91 14 13 12 11

Mandatory Test Cases

Test Case 1

KEYWORD

void push(int data, mystack* ms)

Test Case 2

KEYWORD

int pop(mystack* ms)

Test Case 3

KEYWORD

void merge(mystack* ms1, mystack* ms2)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

435

Test Case 3

NLOC

90

```
Stack2.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 typedef struct mystack {
10     Node* top;
11 } mystack;
12
13 mystack* createStack() {
14     mystack* ms = (mystack*)malloc(sizeof(mystack));
15     ms->top = NULL;
16     return ms;
17 }
18
19 void push(int data, mystack* ms) {
20     Node* temp = (Node*)malloc(sizeof(Node));
21     temp->data = data;
22     temp->next = ms->top;
23     ms->top = temp;
24 }
25
```

```

26 int pop(mystack* ms) {
27     if (ms->top == NULL) return -1;
28     Node* temp = ms->top;
29     int val = temp->data;
30     ms->top = ms->top->next;
31     free(temp);
32     return val;
33 }
34
35 void merge(mystack* ms1, mystack* ms2) {
36     if (ms1->top == NULL) {
37         ms1->top = ms2->top;
38         return;
39     }
40     Node* curr = ms1->top;
41     while (curr->next != NULL) {
42         curr = curr->next;

```

main | CH.SC.U4CSE25225 | Stack2.c

Ln 75, Col 1 Spaces: 4 UTF-8 CRLF C

```
43     }
44     curr->next = ms2->top;
45 }
46
47 int main() {
48     int n, m;
49     if (scanf("%d %d", &n, &m) != 2) return 0;
50
51     mystack* ms1 = createStack();
52     mystack* ms2 = createStack();
53
54     for (int i = 0; i < n; i++) {
55         int val;
56         scanf("%d", &val);
57         push(val, ms1);
58     }
59     for (int i = 0; i < m; i++) {
```

```
60     int val;
61     scanf("%d", &val);
62     push(val, ms2);
63 }
64
65 merge(ms1, ms2);
66
67 Node* curr = ms1->top;
68 while (curr != NULL) {
69     printf("%d ", curr->data);
70     curr = curr->next;
71 }
72 printf("\n");
73 return 0;
74}
75
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 8

9 8 7 6 5

5 6 7 8 3 11 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe

5 6 7 8 9 3 2 11 3 8 7 6 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

10 4

91 18 17 16 15 15 16 17 18 13

11 12 13 14

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe

13 18 17 16 15 15 16 17 18 91 14 13 12 11

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem Description:

You are given an array A of Q integers and Q queries. In each query, you are given an integer i ($1 \leq i \leq N$).

Your task is to find the minimum index greater than i ($1 \leq i \leq N$) such that:

1. Sum of digits of A_i is greater than the sum of digits of A_j
2. $A_i < A_j$

If there is no answer, then print -1.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq A_i \leq 10^9$$

$$1 \leq Q_i \leq N$$

Input format

- The first line contains two numbers N and Q .
- The next line contains N numbers.
- Next Q lines contain Q queries.

Output format

Print the answer as described in the problem

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 5
62 70 28 62 92
1
5
3
4
2

EXPECTED OUTPUT

2 -1 4 -1 -1

Test Case 2

INPUT (STDIN)

6 6
12 71 21 16 91
11
15
31
14
21

EXPECTED OUTPUT

-1 -1 -1 -1 -1 -1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

if(arr[x]<arr[y])

Test Case 2

KEYWORD

if(arr2[x]>arr2[y])

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

10

Test Case 2

TOKEN COUNT

310

Test Case 3

NLOC

57

Stack3.c +

```
1 #include <stdio.h>
2
3 int sumOfDigits(int n) {
4     int sum = 0;
5     while (n > 0) {
6         sum += n % 10;
7         n /= 10;
8     }
9     return sum;
10}
11
12int main() {
13    int N, Q;
14    if (scanf("%d %d", &N, &Q) != 2) return 0;
15
16    int arr[N + 1];
17    int arr2[N + 1];
18    for (int i = 1; i <= N; i++) {
19        scanf("%d", &arr[i]);
20        arr2[i] = sumOfDigits(arr[i]);
```

main | CH.SC.U4CSE25225 | Stack3.c

Ln 40, Col 1 Spaces: 4 UTF-8 CRLF C

```
22
23    for (int q = 0; q < Q; q++) {
24        int x;
25        scanf("%d", &x);
```

```
26     int ans = -1;
27     for (int y = x + 1; y <= N; y++) {
28         if (arr[x] < arr[y]) {
29             if (arr2[x] < arr2[y]) {
30                 ans = y;
31                 break;
32             }
33         }
```

```
34     }
35     printf("%d ", ans);
36 }
37 printf("\n");
38 return 0;
39}
40
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 5

62 70 28 62 92

1

5

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack3.exe

5 -1 5 5 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

6 6

12 71 21 16 91

11

15

31

14

21

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack3.exe

-1 -1 -1 -1 -1 -1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem Description:

Dr. Malar was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Dr. Malar decided to book one ticket for out of those persons also along with his family members.

she wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, implement two stack operations in an array

Can you help them to complete the task?

Constraints

$0 < n < 5$ only five elements has to be practiced for this operation

first element pushed into stack1

second element pushed into stack2, likewise elements pushed into alternative stacks vice versa.

Function Description

- Create a data structure *twoStacks* that represents two stacks.
- Implementation of *twoStacks* should use only one array, i.e., both stacks should use the same array for storing elements.
- Following functions must be supported by *twoStacks*.
 - push1(int x) → pushes x to first stack
 - push2(int x) → pushes x to second stack
 - pop1() → pops an element from first stack and return the popped element

pop2() -> pops an element from second stack and return the popped element
Implementation of *twoStack* should be space efficient.

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

Logical Test Cases

Test Case 1

INPUT (STDIN)

1
2
3
4
5

EXPECTED OUTPUT

Popped element from stack1 is:5
Popped element from stack2 is:4

Test Case 2

INPUT (STDIN)

5
4
5
3
4

EXPECTED OUTPUT

Popped element from stack1 is:4
Popped element from stack2 is:3

Mandatory Test Cases

Test Case 1

KEYWORD

void push1(int x)

Test Case 2

KEYWORD

void push2(int x)

Test Case 3

KEYWORD

int pop1()

Test Case 4

KEYWORD

```
int pop2()
```

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

380

Test Case 3

NLOC

81

Stack4.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX_SIZE 10
5
6 int arr[MAX_SIZE];
7 int top1 = -1;
8 int top2 = MAX_SIZE;
9
10 void push1(int x) {
11     if (top1 < top2 - 1) {
12         arr[++top1] = x;
13     }
14 }
15
16 void push2(int x) {
17     if (top1 < top2 - 1) {
18         arr[--top2] = x;
19     }
20 }
21
22 int pop1() {
23     if (top1 >= 0) {
24         return arr[top1--];
25     }
```

```
26     return -1;
27 }
28
29 int pop2() {
30     if (top2 < MAX_SIZE) {
31         return arr[top2++];
32     }
33     return -1;
```

main | CH.SC.U4CSE25225 | Stack4.c

Ln 66, Col 1 Spaces: 4 UTF-8 CRLF C

```
35
36 int main() {
37     int val;
38     int inputs[20];
39     int count = 0;
```

```
40 while (scanf("%d", &val) == 1) {
41     inputs[count++] = val;
42 }
43
44 if (count == 0) {
45
46     push1(5);
47     push2(4);
48     printf("Popped element from stack1 is:%d\n", pop1());
49     printf("Popped element from stack2 is:%d\n", pop2());
50     return 0;
51 }
52
53 for (int i = 0; i < count; i++) {
```

```
54     if (i % 2 == 0) push1(inputs[i]);
55     else push2(inputs[i]);
56 }
57
58 if (top1 >= 0) {
59     printf("Popped element from stack1 is:%d\n", pop1());
60 }
61 if (top2 < MAX_SIZE) {
62     printf("Popped element from stack2 is:%d\n", pop2());
63 }
64 return 0;
65}
66
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

1
2
3
4
5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack4.exe

Popped element from stack1 is:5

Popped element from stack2 is:4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5
4
5
3
4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack4.exe

Popped element from stack1 is:4

Popped element from stack2 is:3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Stack	Question Information	Level 1	Challenge 45
----	---------	-------	----------------------	---------	--------------

Problem Description:

Sajid was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Sajid decided to book one ticket for out of those persons also along with his family members.

He wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, Check for Balanced Brackets in an expression using Stack

Can you help them to complete the task?

Function Description

- Declare a character stack S.
- Now traverse the expression string exp.
 1. If the current character is a starting bracket ('{' or '[' or '{') then push it to stack.
 2. If the current character is a closing bracket (')' or ']' or '}') then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then “not balanced”

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

{()}[]

EXPECTED OUTPUT

Balanced

Test Case 2

INPUT (STDIN)

{{{}}}}

EXPECTED OUTPUT

Not Balanced

▼ Mandatory Test Cases

Test Case 1

KEYWORD

pop()

Test Case 2

KEYWORD

empty()

Test Case 3

KEYWORD

push

Test Case 4

KEYWORD

stack

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

15

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

100

```
Stack5.c +
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdbool.h>
4
5 #define MAX 1000
6
7 char stack[MAX];
8 int top = -1;
9
10 void push(char c) {
11     if (top < MAX - 1) {
12         stack[++top] = c;
13     }
14 }
15
16 char pop() {
17     if (top >= 0) {
18         return stack[top--];
19     }
20     return '\0';
21 }
22
23 bool isMatchingPair(char char1, char char2) {
24     if (char1 == '(' && char2 == ')') return true;
25     if (char1 == '{' && char2 == '}') return true;
```

```
26     if (char1 == '[' && char2 == ']') return true;
27     return false;
28 }
29
```

main | CH.SC.U4CSE25225 | Stack5.c

Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30 int main() {
31     char exp[MAX];
32     if (scanf("%s", exp) != 1) return 0;
33
34     bool balanced = true;
35     for (int i = 0; exp[i] != '\0'; i++) {
36         if (exp[i] == '{' || exp[i] == '(' || exp[i] == '[') {
```

```
37         push(exp[i]);
38     } else if (exp[i] == '{' || exp[i] == '(' || exp[i] == '[') {
39         if (top == -1 || !isMatchingPair(pop(), exp[i])) {
40             balanced = false;
41             break;
42         }
43     }
44 }
45
46 if (top != -1) {
47     balanced = false;
```

```
48     }
49
50     if (balanced) {
51         printf("Balanced\n");
52     } else {
53         printf("Not Balanced\n");
54     }
55     return 0;
56 }
57
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
Balanced
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack5.exe
```

```
Balanced
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
{{{}}]}}
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack5.exe
```

```
Not Balanced
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 46
----	---------	-------	----------------------	---------	--------------

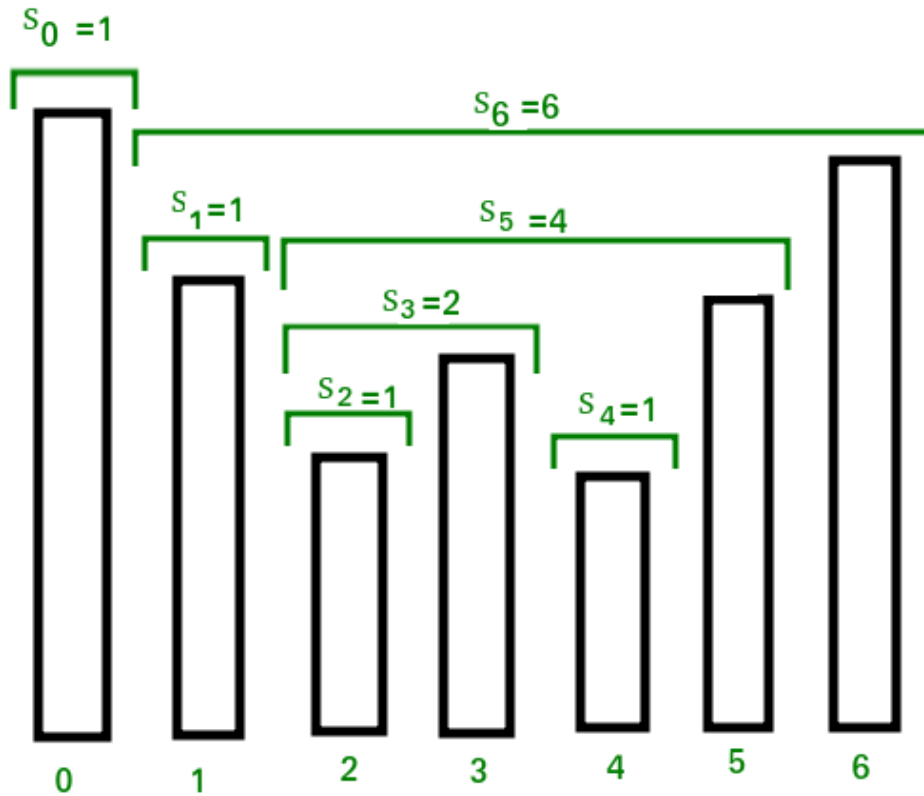
Question description

The stock span problem is a financial problem where we have a series of n daily price quotes for a stock and we need to calculate span of stock’s price for all n days.

The span S_i of the stock’s price on a given day i is defined as the maximum number of consecutive days just before the given day, for which the price of the stock on the current day is less than or equal to its price on the given day.

For example, if an array of 7 days prices is given as {100, 80, 60, 70, 60, 75, 85},

then the span values for corresponding 7 days are {1, 1, 1, 2, 1, 4, 6}



Constraints

$0 < n < 1000$

$0 < \text{price}[i] < 1000000$

Input Format:

First line indicates the number of days

second line indicates the price quoted for above mentioned days

Output Format:

Single line represents the span values for corresponding days

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

6
9 8 7 6 5 4

EXPECTED OUTPUT

1 1 1 1 1 1

Test Case 2

INPUT (STDIN)

6
10 41 15 90 10 70

EXPECTED OUTPUT

1 2 1 4 1 2

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void printArray(int
arr[],int n)

Test Case 2

KEYWORD

void calculateSpan(int
price[],int n,int S[])

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

220

Test Case 3

NLOC

35

```
Stack6.c +
1 #include <stdio.h>
2
3 void calculateSpan(int price[], int n, int S[]) {
4     int st[n];
5     int top = -1;
6
7     st[++top] = 0;
8     S[0] = 1;
9
10    for (int i = 1; i < n; i++) {
11        while (top >= 0 && price[st[top]] <= price[i]) {
12            top--;
13        }
14        S[i] = (top == -1) ? (i + 1) : (i - st[top]);
15        st[++top] = i;
16    }
17}
18
19void printArray(int arr[], int n) {
20    for (int i = 0; i < n; i++) {
21        printf("%d%c", arr[i], (i == n - 1) ? '\n' : ' ');
22    }
23}
24
25int main() {
```

Ln 37, Col 1 Spaces: 4 UTF-8 CRLF C

```
26  int n;  
27  if (scanf("%d", &n) != 1) return 0;  
28  int price[n];  
29  for (int i = 0; i < n; i++) {  
30      scanf("%d", &price[i]);  
31  }
```

```
32  int S[n];
33  calculateSpan(price, n, S);
34  printArray(S, n);
35  return 0;
36}
37
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

6

9 8 7 6 5 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack6.exe

1 1 1 1 1 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

6

10 41 15 90 10 70

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack6.exe

1 2 1 4 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

You are given an array A of N integers. Now, two functions $F(X)$ and $G(X)$ are defined:

$F(X)$: This is the smallest number Z such that $X < Z \leq N$ and $A[X] < A[Z]$

$G(X)$: This is the smallest number Z such that $X < Z \leq N$ and $A[X] > A[Z]$

Now, you need to find for each index i of this array $G(F(i))$, where $1 \leq i \leq N$. If such a number does not exist, for particular index i , output 1 as its answer. If such a number does exist, output $A[G(F(i))]$

Constraints:

$$1 \leq N \leq 3000$$

$$0 \leq A[i] \leq 10^{18}$$

Input :

The first line contains a single integer N denoting the size of array A . Each of the next N lines contains a single integer, where the integer on the i^{th} line denotes $A[i]$.

Output :

Print N space-separated integers on a single line, where the i^{th} integer denotes $A[G(F(i))]$ or 1 , if $G(F(i))$ does not exist.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

8
3 7 1 7 8 4 5 2

EXPECTED OUTPUT

1 4 4 4 -1 2 -1 -1

Test Case 2

INPUT (STDIN)

8
31 71 11 17 81 14 51 21

EXPECTED OUTPUT

11 14 14 14 -1 21 -1 -1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

if(a[j]>a[i])

Test Case 2

KEYWORD

for(i=0;i<n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

225

Test Case 3

NLOC

30

```
Stack7.c +
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1) return 0;
6     long long a[n];
7     for (int i = 0; i < n; i++) {
8         scanf("%lld", &a[i]);
9     }
10
11     int next_greater[n];
12     for (int i = 0; i < n; i++) {
13         next_greater[i] = -1;
14         for (int j = i + 1; j < n; j++) {
15             if (a[j] > a[i]) {
16                 next_greater[i] = j;
17                 break;
18             }
19         }
20     }
21
22     int next_smaller[n];
23     for (int i = 0; i < n; i++) {
24         next_smaller[i] = -1;
25         for (int j = i + 1; j < n; j++) {
```

```
26         if (a[j] < a[i]) {
27             next_smaller[i] = j;
28             break;
29         }
30     }
31 }
32
33 int i;
34 for (i = 0; i < n; i++) {
35     int fg = -1;
36     int f = next_greater[i];
```

```
37     if (f != -1) {
38         int g = next_smaller[f];
39         if (g != -1) {
40             fg = a[g];
41         }
42     }
43     printf("%d%c", fg, (i == n - 1) ? '\n' : ' ');
44 }
45 return 0;
46}
47
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

8

3 7 1 7 8 4 5 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack7.exe

1 4 4 4 -1 2 -1 -1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

31 71 11 17 81 14 51 21

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack7.exe

11 14 14 14 -1 21 -1 -1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

Hassan enjoys jumping from one building to the next. However, he merely jumps to the next higher building and stops when there are none accessible. The amount of stamina necessary for a voyage is equal to the xor of all the heights Hassan leaps till he comes to a halt.

If heights are [1 2 4], and he starts from 1, goes to 2 stamina required is $1 \oplus 2 = 3$, then from 2 to 3. Stamina for the entire journey is $1 \oplus 2 \oplus 4 = 7$. Find the maximum stamina required if can start his journey from any building.

Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq \text{Height} \leq 10^9$$

Input

First line: N , no of buildings.

Second line: N integers, defining heights of buildings.

Output

Single Integer is the maximum stamina required for any journey.

Explanation:

8

1 2 3 8 6 4 7 9

Considering the input given by you, 1st starting point is 1. From this building, Hassan will try to find the next higher building that is building 2 here and so on. Likewise he will complete all pass.

Pass 1: 1-> 2->3->8-> 9, XOR = 1

Pass 2: 2->3->8-> 9, XOR = 0

Pass 3: 3->8-> 9, XOR = 2

Pass 4: 8-> 9, XOR = 1

Pass 5: 6->7->9 , XOR = 8

Pass 6: 4->7->9 , XOR = 10

Pass 7: 7-> 9 XOR = 14

Pass 8: 9 XOR = 9

So, the highest XOR value is 14

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
1 2 3 8 6

EXPECTED OUTPUT

11

Test Case 2

INPUT (STDIN)

8
1 2 3 8 6 4 7 9

EXPECTED OUTPUT

14

✓ Mandatory Test Cases

Test Case 1

KEYWORD

```
arr[1000000]
```

Test Case 2

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
if(arr[i]<arr[j])
```

Test Case 4

KEYWORD

```
st[i]=arr[i]^st[j];
```

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

205

Test Case 3

NLOC

26

```
Stack8.c +
1 #include <stdio.h>
2
3 int arr[1000000];
4 int st[1000000];
5
6 int main() {
7     int n;
8     if (scanf("%d", &n) != 1) return 0;
9
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &arr[i]);
12    }
13
14    int max_stamina = 0;
15    int i, j;
16
17    for (i = n - 1; i >= 0; i--) {
18        int next_higher = -1;
19        for (j = i + 1; j < n; j++) {
20            if (arr[i] < arr[j]) {
21                next_higher = j;
22                break;
23            }
24        }
25        if (next_higher != -1) {
```

main | CH.SC.U4CSE25225 | Stack8.c

Ln 39, Col 1 Spaces: 4 UTF-8 CRLF C

```
26         j = next_higher;
27         st[i] = arr[i] ^ st[j];
28     } else {
29         st[i] = arr[i];
30     }
31     if (st[i] > max_stamina) {
32         max_stamina = st[i];
```

```
33     }  
34 }  
35  
36 printf("%d\n", max_stamina);  
37 return 0;  
38}  
39
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

1 2 3 8 6

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack8.exe

11

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 2 3 8 6 4 7 9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack8.exe

14

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

Rathik organized technical round interview in Macrosoft for the set of computer science candidates.

The problem is to perform Implement a stack using single queue. you have to use queue data structure, the task is to implement stack using only given queue data structure.

Rathik have given the deadline of only 5 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit ?

Function Description

x is the element to be pushed and s is stack

push(s, x)

- 1) Let size of q be s.
- 2) Enqueue x to q
- 3) One by one Dequeue s items from queue and enqueue them.

Removes an item from stack

pop(s)

1) Dequeue an item from q

Constraints

$0 < n, m < N$

$1 < \text{arr}[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into stack and m is the number of pop operation need to be performed

next line indicates the n number stack elements

Output Format:

First line indicates top of the element of the stack

second line indicates the top of the element after the pop operation

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
1 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 3
```

Test Case 2

INPUT (STDIN)

```
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 11
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void Stack::push(int val)

Test Case 4

KEYWORD

q.pop();

Test Case 2

KEYWORD

q.push(val)

Test Case 3

KEYWORD

void Stack::pop()

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

320

Test Case 3

NLOC

60

Code Editor

c

```
1 #include <stdio.h>
2 int main()
3 {
4
```

Custom Input (stdin)

T1T2

Type Here

Stack9.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 1000
5
6 int q[MAX];
7 int front = 0;
8 int rear = 0;
9
10 void q_push(int val) {
11     q[rear++] = val;
12 }
13
14 int q_pop() {
15     return q[front++];
16 }
17
18 int q_size() {
19     return rear - front;
20 }
21
22 int q_front() {
23     return q[front];
24 }
25
```

```
26 void push(int val) {  
27     int s = q_size();  
28     q_push(val);  
29     for (int i = 0; i < s; i++) {  
30         q_push(q_pop());  
31     }  
32 }  
33
```

main | CH.SC.U4CSE25225 | Stack9.c

Ln 66, Col 1 Spaces: 4 UTF-8 CRLF C

```
35     if (q_size() > 0) {  
36         q_pop();  
37     }  
38 }  
39
```

```
40 int top() {
41     if (q_size() > 0) {
42         return q_front();
43     }
44     return -1;
45 }
46
47 int main() {
48     int n, m;
49     if (scanf("%d %d", &n, &m) != 2) return 0;
50
51     for (int i = 0; i < n; i++) {
52         int val;
53         scanf("%d", &val);
```

```
54     push(val);
55 }
56
57 printf("top of element %d\n", top());
58
59 for (int i = 0; i < m; i++) {
60     pop();
61 }
62
63 printf("top of element %d\n", top());
64 return 0;
65}
66
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 2
```

```
1 2 3 4 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack9.exe
```

```
top of element 5
```

```
top of element 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
15 4
```

```
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack9.exe
```

```
top of element 5
```

```
top of element 11
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Stack	Question Information	Level 1	Challenge 50
----	---------	-------	----------------------	---------	--------------

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
 - Create a string by concatenating the two operands and the operator after them.
 - string = operand1 + operand2 + operator**
 - And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfix expressions

Output Format

Single line represents the prefix expression

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

ABC/-AK/L-*

EXPECTED OUTPUT

*-A/BC-/AKL

Test Case 2

INPUT (STDIN)

AB+CD-*

EXPECTED OUTPUT

*+AB-CD

▼ Mandatory Test Cases

Test Case 1

KEYWORD

string postToPre(string
post_exp)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

220

Test Case 3

NLOC

46

Stack10.c +

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 #include <stdlib.h>
5
6 #define MAX 1000
7
8 typedef struct {
9     char str[MAX];
10 } StackItem;
11
12 StackItem stack[MAX];
13 int top = -1;
14
15 void push(char *s) {
16     top++;
17     strcpy(stack[top].str, s);
18 }
19
20 void pop(char *s) {
21     if (top >= 0) {
22         strcpy(s, stack[top].str);
23         top--;
24     }
25 }
```

```
26
27int isOperator(char c) {
28    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
29}
30
31void postToPre(char *post_exp, char *result) {
32    top = -1;
33    int len = strlen(post_exp);
34
35    for (int i = 0; i < len; i++) {
36        char c = post_exp[i];
37        if (!isOperator(c)) {
38            char temp[2] = {c, '\0'};
39            push(temp);
40        } else {
```

```
41         if (top >= 1) {
42             char op2[MAX], op1[MAX];
43             pop(op2);
44             pop(op1);
45
46             char combined[MAX];
47             combined[0] = c;
48             combined[1] = '\0';
49             strcat(combined, op1);
50             strcat(combined, op2);
51
52             push(combined);
53         }
54     }
55 }
```

main | CH.SC.U4CSE25225 | Stack10.c | Ln 83, Col 1 | Spaces: 4 | UTF-8 | CRLF | C

```
56
57     while (top > 0) {
58         char op2[MAX], op1[MAX];
59         pop(op2);
60         pop(op1);
61         char combined[MAX];
62         strcpy(combined, op1);
63         strcat(combined, op2);
64         push(combined);
65     }
66
67     if (top >= 0) {
68         strcpy(result, stack[top].str);
69     } else {
```

```
70     result[0] = '\0';
71 }
72}
73
74int main() {
75     char post_exp[MAX];
76     if (scanf("%s", post_exp) != 1) return 0;
77
78     char pre_exp[MAX];
79     postToPre(post_exp, pre_exp);
80     printf("%s\n", pre_exp);
81     return 0;
82}
83
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
ABC/-AK/L-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*-A/BC-/AKL
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
AB+CD-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*+AB-CD
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```